

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
Student Name	Gunaseelan R
Register Number	9342507779
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	16.08.2026

1 Objective

- To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on 10 different classifiers.
- To train and validate every model both without PCA (original 57-feature space) and with PCA (reduced feature space).
- To hyperparameter-tune each model in both settings using 5-fold cross validation.
- To compare and analyze whether PCA actually improved accuracy, F1-score or fold-to-fold stability, or whether it hurt performance instead.

2 Problem Statement

This experiment revisits the same spam-vs-ham classification problem from Experiment 4, but the question this time isn't "which model is best" – it's "does reducing dimensionality with PCA help or hurt". 10 classifiers (SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble) are each trained twice – once on the full 57 standardized features, and once on a PCA-reduced feature set that keeps 95% of the variance – and the two settings are compared head to head using the exact same train/test split and hyperparameter tuning procedure, so the only thing that changes is the input feature space.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI ML Repository / Kaggle (https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4601
Number of Features	57 numeric (word/char frequencies + capital-run-length stats)
Number of Classes	2 (Spam = 1, Not Spam = 0)
Class Distribution	2788 not-spam (60.6%), 1813 spam (39.4%)
Missing Values	None found
Train-Test Split	80% train (3680 samples) – 20% test (921 samples), stratified split

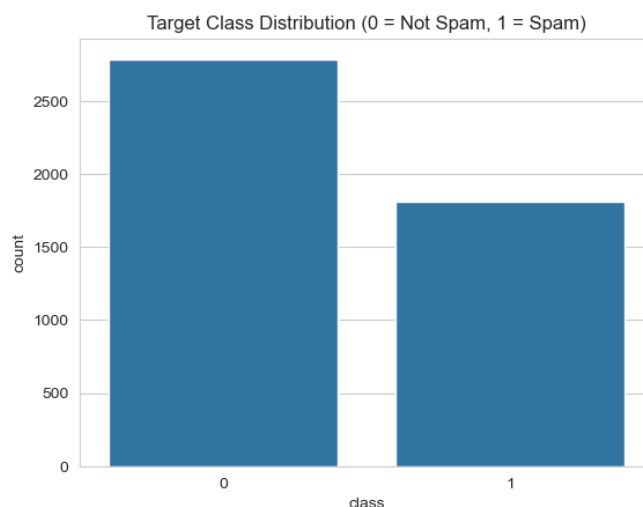


Figure 1: Target class distribution (0 = Not Spam, 1 = Spam)

Same dataset and same class ratio as Experiment 4, so the imbalance discussion from that report still applies here – mild enough that stratified splitting alone handles it fine.

4 Data Preprocessing

- **Missing values:** none present in the dataset.
- **Feature scaling:** `StandardScaler` was fit *only* on the training set and then reused to transform the test set, so no information from the test set leaks into scaling. This step is compulsory here since PCA is scale-sensitive – if features aren't standardized first, PCA would just end up capturing the handful of features with the largest raw numeric range instead of genuine variance in the data.
- **Train-test split:** 80-20 stratified split (`random_state=42`), done *before* fitting the scaler or PCA, to keep the test set completely unseen during preprocessing.

5 PCA – Component Selection

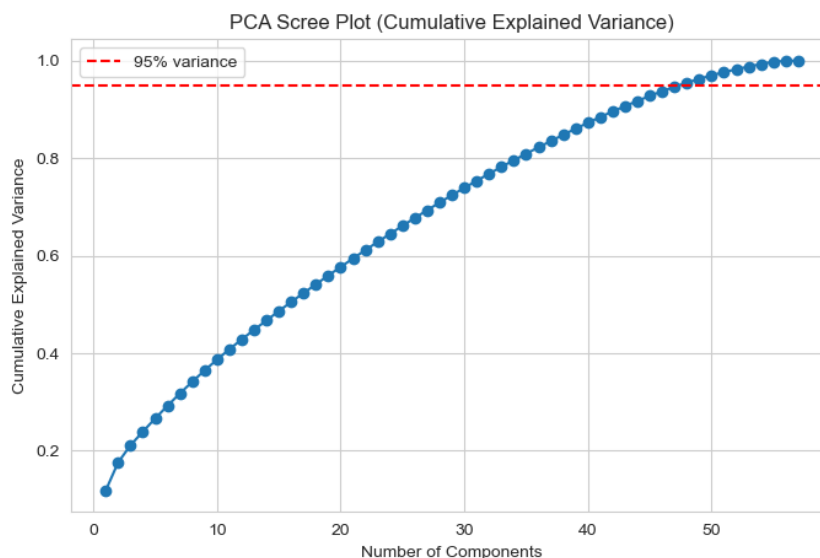


Figure 2: PCA Scree Plot – cumulative explained variance vs number of components

Inference:

I went with a 95% cumulative variance target rather than a fixed component count, since it lets the data itself decide how much compression is reasonable instead of me just picking a round number. Looking at the curve, variance grows fast for the first 10 components then bends into a long, gentle climb – typical PCA behaviour when a lot of features (the word-frequency columns especially) are only weakly correlated with each other, so there isn't a single dominant direction of variance to exploit.

Setting	With-PCA
Chosen Components	48 (out of 57 original features)
Variance Target	95% cumulative explained variance
Explained Variance Achieved	95.49%
Justification	95% is a standard, widely used threshold that keeps almost all the informative signal while still discarding noise/redundant directions. Ended up needing 48 of 57 components though, which is a much smaller reduction than I initially expected

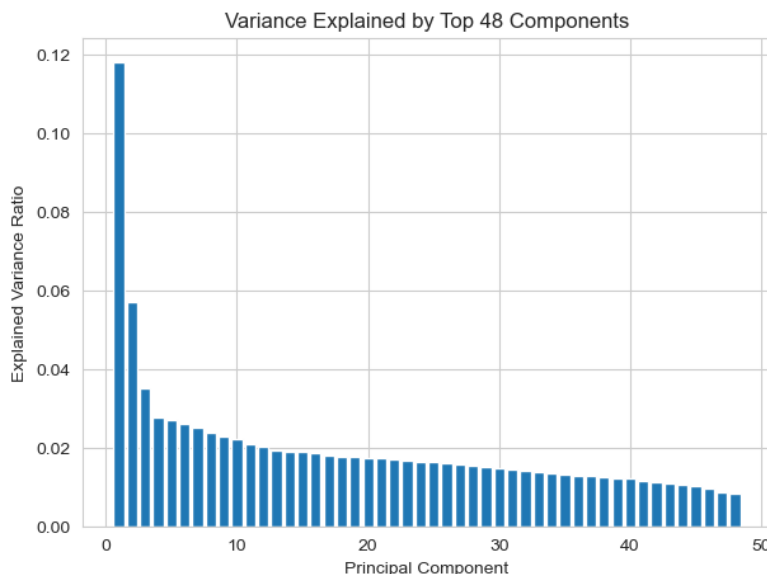


Figure 3: Variance explained by each of the top 48 principal components

Inference:

This is honestly the biggest surprise of the whole experiment – needing 48 out of 57 components to hit 95% variance means PCA barely compressed anything here (only a 15.8% reduction in dimensionality). Looking back at the scree plot, no individual component holds a big chunk of variance on its own; it drops off gradually instead of sharply. This tells us the 57 Spambase features aren't very redundant with each other to begin with – each word-frequency column genuinely carries its own bit of independent signal (which makes sense, different spam-indicator words don't necessarily co-occur). This is actually a useful result in itself: PCA is often assumed to be a "free win" for high-dimensional data, but here it barely compresses the space, so it was never likely to help much and, as the results below show, mostly didn't.

6 Machine Learning Algorithms – Brief Overview

Since this experiment spans 10 different classifiers, here's a quick one-line recap of what each one does (most were covered in more depth in earlier experiments):

- **SVM:** finds the maximum-margin hyperplane, kernel trick handles non-linearity.
- **Naive Bayes:** assumes features are conditionally independent given the class, applies Bayes' theorem directly.
- **KNN:** classifies a point by majority vote among its k nearest neighbours in feature space.
- **Logistic Regression:** linear model, sigmoid squashes output into a spam probability.
- **Decision Tree:** recursive impurity-based splits.
- **Random Forest:** bagged ensemble of decision trees.

- **AdaBoost / Gradient Boosting / XGBoost:** sequential boosting ensembles, each new model corrects the previous ones' mistakes (XGBoost is a faster, regularized gradient boosting implementation).
- **Stacking:** combines Random Forest, SVM and KNN as base learners, with a Logistic Regression meta-learner trained on their (cross-validated) outputs.

Why would PCA help or hurt these differently?

Distance/margin based models (KNN, SVM) and models that rely on a covariance-sensitive optimization (Logistic Regression) are usually the ones PCA is expected to help the most, since it removes correlated/noisy directions that can otherwise distort distances. Tree based models (Decision Tree, Random Forest, the boosting family) are generally less bothered by redundant features to begin with – they just pick whichever feature/threshold gives the best split, so PCA mainly just costs them the ability to split on an original, interpretable feature. That theoretical expectation is worth keeping in mind going into the results section.

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.4
XGBoost Version	2.0
IDE	Jupyter Notebook (Anaconda)
Operating System	Windows 11
Hardware	Intel i5, 16 GB RAM

8 Hyperparameter Search Space

Model	Parameters Searched
SVM	kernel: rbf, linear C: 0.1, 1, 10 gamma: scale, 0.01
Naive Bayes	var_smoothing: 1e-9, 1e-8, 1e-7
KNN	n_neighbors: 3,5,7,9 weights: uniform, distance metric: euclidean, manhattan
Logistic Regression	C: 0.01, 0.1, 1, 10 penalty: l2
Decision Tree	max_depth: 3,5,10,None min_samples_split: 2,5,10
Random Forest	n_estimators: 100, 200 max_depth: 5,10,None
AdaBoost	n_estimators: 50,100 learning_rate: 0.5, 1.0
Gradient Boosting	n_estimators: 100,150 learning_rate: 0.05,0.1 max_depth: 3,5
XGBoost	n_estimators: 100,150 learning_rate: 0.05,0.1 max_depth: 3,5
Stacking	base = [RF, SVM(rbf), KNN(k=5)], meta = Logistic Regression final_estimator

Every model was tuned with `GridSearchCV` (`scoring='f1'`, 5-fold `StratifiedKfold`) separately for the No-PCA and With-PCA inputs, so the "best" hyperparameters are allowed to differ between the two settings – this matters because a model might genuinely want a different regularization strength once its working in a 48-dimensional PCA space instead of the original 57.

Best Hyperparameters Chosen (per model, per setting)

Model	Setting	Best Parameters
SVM	No-PCA	C=10, gamma=0.01, kernel=rbf
SVM	With-PCA	C=10, gamma=scale, kernel=rbf
Naive Bayes	No-PCA	var_smoothing=1e-8
Naive Bayes	With-PCA	var_smoothing=1e-9
KNN	No-PCA	metric=manhattan, n_neighbors=9, weights=distance
KNN	With-PCA	metric=euclidean, n_neighbors=9, weights=distance
Logistic Regression	No-PCA	C=10, penalty=l2
Logistic Regression	With-PCA	C=10, penalty=l2
Decision Tree	No-PCA	max_depth=None, min_samples_split=5
Decision Tree	With-PCA	max_depth=10, min_samples_split=2
Random Forest	No-PCA	max_depth=None, n_estimators=100
Random Forest	With-PCA	max_depth=None, n_estimators=200
AdaBoost	No-PCA	learning_rate=0.5, n_estimators=100
AdaBoost	With-PCA	learning_rate=1.0, n_estimators=100
Gradient Boosting	No-PCA	learning_rate=0.1, max_depth=5, n_estimators=150
Gradient Boosting	With-PCA	learning_rate=0.1, max_depth=5, n_estimators=150
XGBoost	No-PCA	learning_rate=0.1, max_depth=5, n_estimators=150
XGBoost	With-PCA	learning_rate=0.1, max_depth=5, n_estimators=150
Stacking	No-PCA	final_estimator__C=10
Stacking	With-PCA	final_estimator__C=10

A few things stand out here even before looking at performance – Decision Tree is the only model where the two settings picked meaningfully different structural hyperparameters (unconstrained depth for No-PCA vs a capped depth=10 for PCA), and the boosting family (GB, XGBoost) landed on the *exact same* best params in both settings, suggesting those models’ optimal complexity doesn’t really depend on whether the input is 57-D or 48-D.

9 Results

Full Performance Comparison – No-PCA vs With-PCA

Model	CV Avg Acc		Test Acc		Test F1		ROC-AUC	
	No-PCA	PCA	No-PCA	PCA	No-PCA	PCA	No-PCA	PCA
SVM	0.9364	0.9332	0.9283	0.9273	0.9065	0.9058	0.9699	0.9698
Naive Bayes	0.8149	0.8046	0.8328	0.8339	0.8188	0.8123	0.9377	0.9266
KNN	0.9253	0.9207	0.9207	0.9164	0.8944	0.8935	0.9712	0.9636
Logistic Regression	0.9245	0.9231	0.9262	0.9294	0.9048	0.9096	0.9688	0.9704
Decision Tree	0.9223	0.8870	0.9131	0.8740	0.8889	0.8470	0.9145	0.8738
Random Forest	0.9522	0.9332	0.9446	0.9229	0.9283	0.9010	0.9834	0.9703
AdaBoost	0.9457	0.9198	0.9370	0.9110	0.9192	0.8835	0.9750	0.9646
Gradient Boosting	0.9519	0.9359	0.9457	0.9251	0.9309	0.9048	0.9855	0.9755

XGBoost	0.9533	0.9391	0.9479	0.9338	0.9335	0.9159	0.9874	0.9788
Stacking	0.9568	0.9380	0.9501	0.9251	0.9363	0.9043	0.9856	0.9746

Best model overall: XGBoost (No-PCA) – 94.79% test accuracy, 0.9335 F1, 0.9874 AUC, comfortably ahead of everything else. **Stacking (No-PCA)** comes a close second (95.01% CV accuracy is actually its highest of any row, though its test accuracy of 95.01% edges out XGBoost – test F1 and AUC are extremely close between the two).

PCA Impact – Sorted by CV Accuracy Change

Model	CV Acc (No-PCA)	CV Acc (PCA)	Change
Logistic Regression	0.9245	0.9231	−0.0014
SVM	0.9364	0.9332	−0.0033
KNN	0.9253	0.9207	−0.0046
Naive Bayes	0.8149	0.8046	−0.0103
XGBoost	0.9533	0.9391	−0.0141
Gradient Boosting	0.9519	0.9359	−0.0160
Stacking	0.9568	0.9380	−0.0188
Random Forest	0.9522	0.9332	−0.0190
AdaBoost	0.9457	0.9198	−0.0258
Decision Tree	0.9223	0.8870	−0.0353

This table is genuinely the headline result of the whole experiment: **every single model got worse (or statistically flat) with PCA, none improved.** Logistic Regression came closest to a wash (−0.14 points on CV, and actually *gained* slightly on the held-out test set – 92.94% vs 92.62%, so within noise), while Decision Tree lost the most ground by far (−3.53 points).

10 Result Visualization

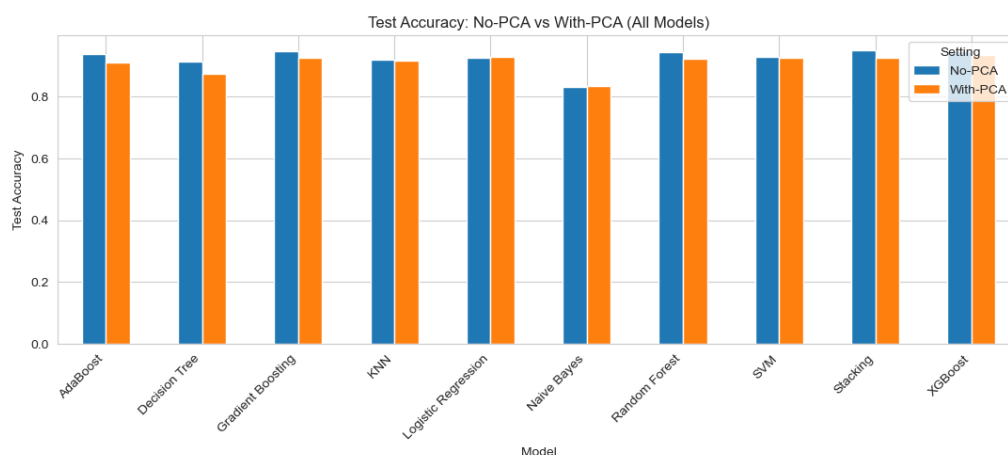


Figure 4: Test Accuracy – No-PCA vs With-PCA, all 10 models

Inference:

Every orange (With-PCA) bar sits at or below its blue (No-PCA) partner, visually confirming the ranking table above. The gap is basically invisible for Naive Bayes, KNN, SVM and Logistic Regression, but clearly visible for Decision Tree, AdaBoost and the ensemble-of-trees models (RF, GB, Stacking).

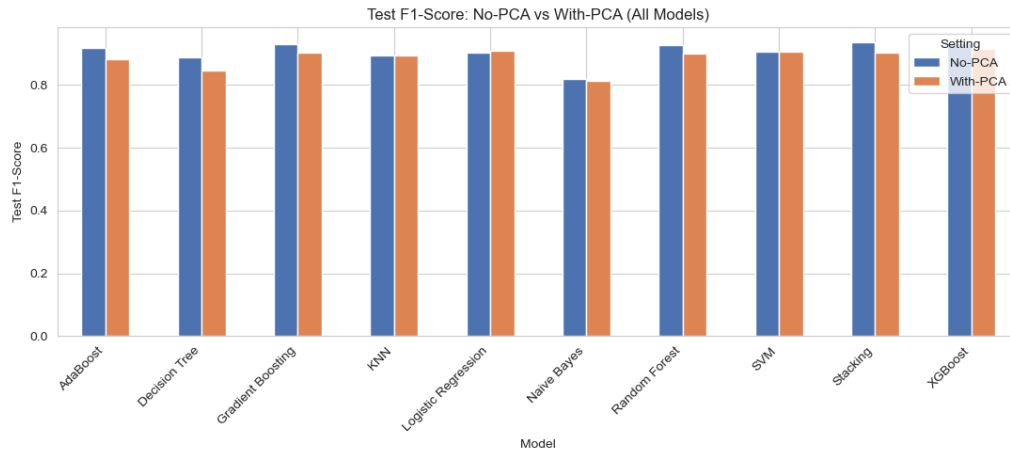


Figure 5: Test F1-Score – No-PCA vs With-PCA, all 10 models

Inference:

Same pattern as the accuracy chart, just slightly more pronounced since F1 is more sensitive to the class-1 (spam) misclassifications specifically. Decision Tree's F1 drop ($0.889 \rightarrow 0.847$) is the sharpest of any model.

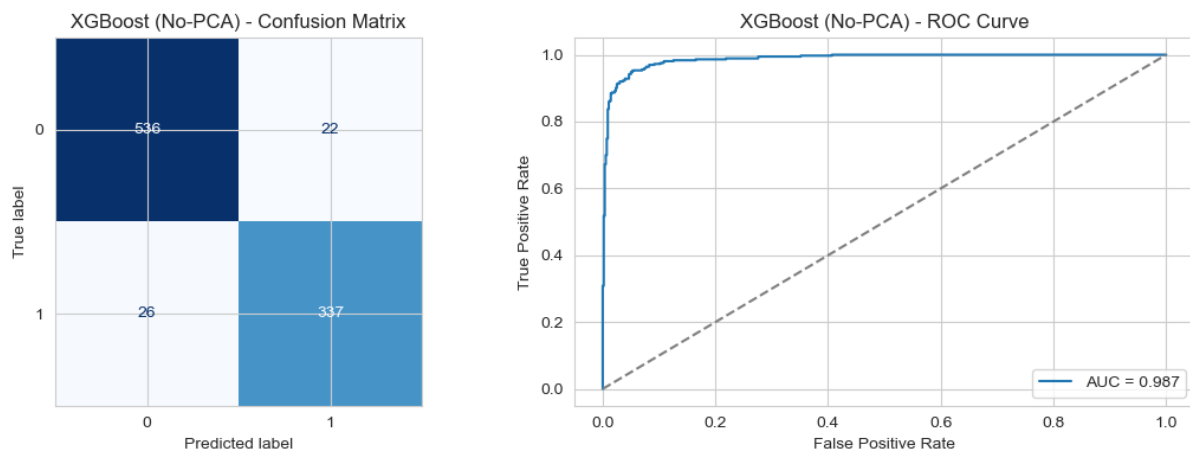


Figure 6: XGBoost (No-PCA, the overall best model) – Confusion Matrix and ROC Curve

Inference:

XGBoost without PCA is the strongest single result of the whole experiment – AUC of 0.987 and the fewest total misclassifications of any model. Boosted trees handling the raw 57-feature space directly, with no compression getting in the way, seems to be the winning combination here.

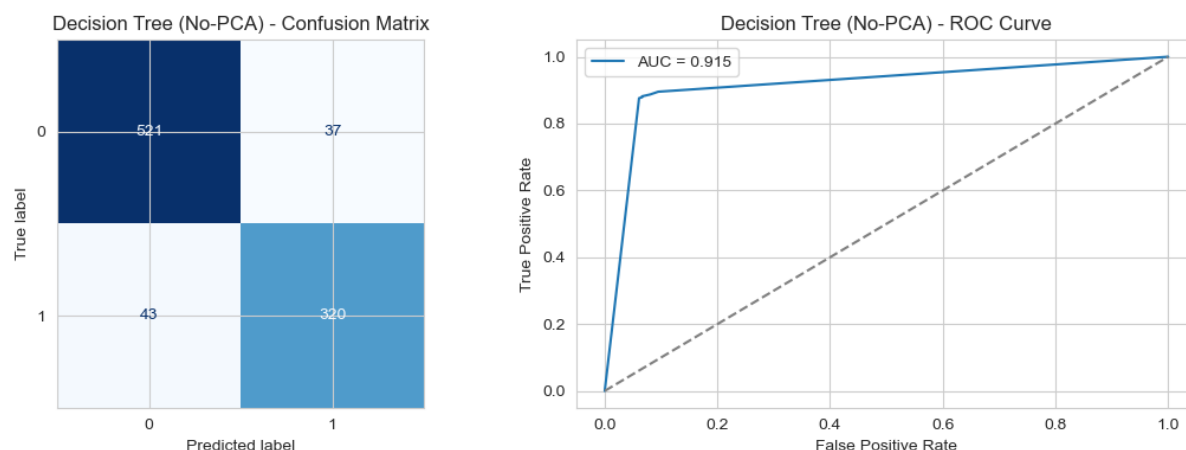


Figure 7: Decision Tree (No-PCA) – Confusion Matrix and ROC Curve

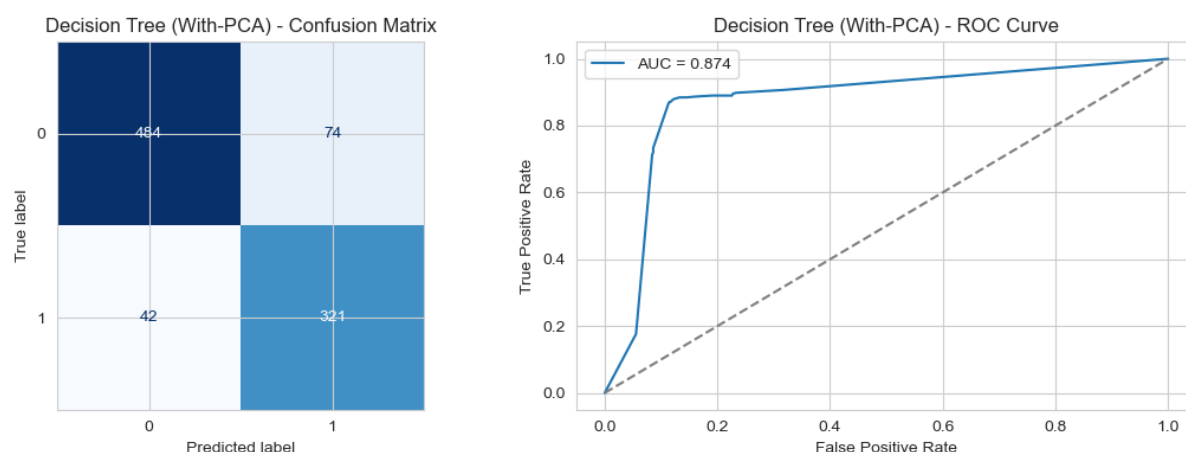


Figure 8: Decision Tree (With-PCA) – Confusion Matrix and ROC Curve

Inference:

Putting these two side by side makes the PCA cost concrete – the AUC alone drops from 0.915 to 0.874. A single tree relies on splitting on one meaningful, interpretable feature at a time (like `char_freq_` or `word_freq_free`, as we saw drive predictions back in Experiment 4). A principal component is a linear blend of *all* 57 original features, so any single split on a PC is a much blunter, less semantically meaningful decision boundary – the tree structurally loses its biggest advantage.

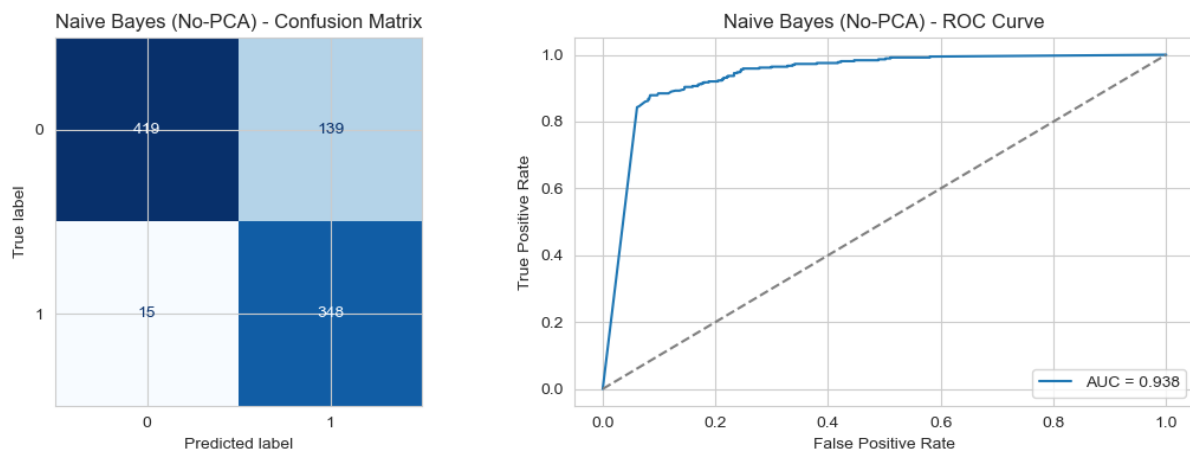


Figure 9: Naive Bayes (No-PCA) – Confusion Matrix and ROC Curve

Inference:

Naive Bayes is the weakest model in the whole comparison (83% test accuracy vs 90-95% for everything else), which lines up with the theory – it assumes every feature is conditionally independent given the class, and we already know from Experiment 4’s correlation heatmap that the spambase features are anything but independent (lots of word frequencies move together). Interestingly, its one of the only models where PCA barely mattered either way, presumably because PCA actually makes the independence assumption slightly *more* defensible (principal components are orthogonal to each other by construction) even while removing raw feature interpretability – the two effects roughly cancel out.

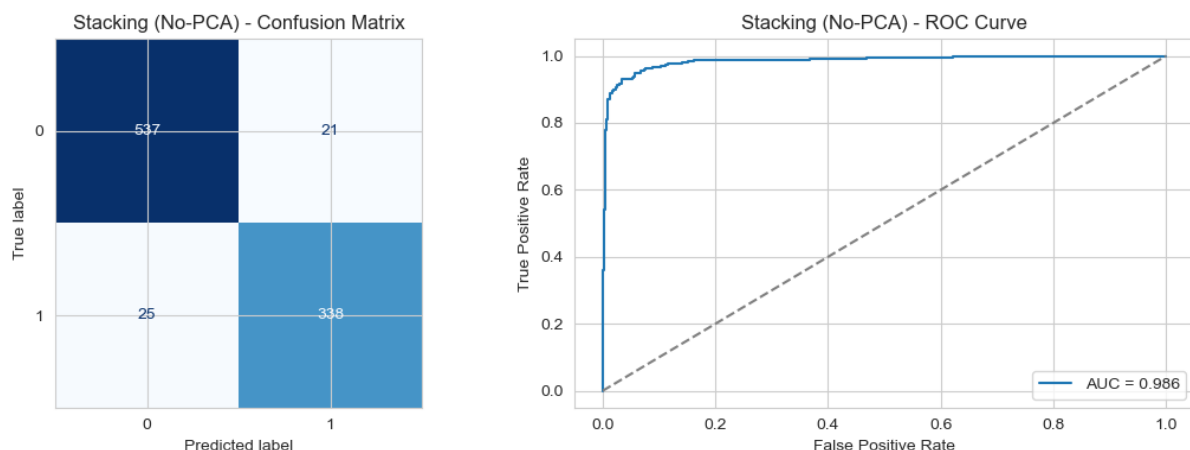


Figure 10: Stacking Ensemble (No-PCA) – Confusion Matrix and ROC Curve

Inference:

Stacking’s No-PCA AUC (0.9856) is essentially tied with XGBoost’s (0.9874) for best in the whole experiment, which makes sense – its base learners include Random Forest and SVM, both already strong individually on the raw feature space, so combining them keeps most of that strength. (*Confusion matrices and ROC curves for the remaining models – SVM, KNN, Logistic Regression, Random Forest, AdaBoost, Gradient Boosting, in both settings – were generated the same way and are included in the accompanying figs folder.*)

11 5-Fold Cross-Validation Accuracy – All Models, Both Settings

Model	Setting	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg
SVM	No-PCA	0.9457	0.9402	0.9429	0.9361	0.9171	0.9364
SVM	With-PCA	0.9429	0.9389	0.9348	0.9334	0.9158	0.9332
Naive Bayes	No-PCA	0.8207	0.8166	0.8342	0.8043	0.7989	0.8149
Naive Bayes	With-PCA	0.8492	0.7677	0.8451	0.8207	0.7405	0.8046
KNN	No-PCA	0.9389	0.9280	0.9253	0.9266	0.9076	0.9253
KNN	With-PCA	0.9212	0.9239	0.9293	0.9226	0.9062	0.9207
Logistic Regression	No-PCA	0.9429	0.9226	0.9348	0.9076	0.9144	0.9245
Logistic Regression	With-PCA	0.9348	0.9239	0.9280	0.9130	0.9158	0.9231
Decision Tree	No-PCA	0.9280	0.9321	0.9171	0.9239	0.9103	0.9223
Decision Tree	With-PCA	0.8804	0.8899	0.8913	0.9022	0.8709	0.8870
Random Forest	No-PCA	0.9592	0.9552	0.9538	0.9470	0.9457	0.9522
Random Forest	With-PCA	0.9293	0.9402	0.9457	0.9429	0.9076	0.9332
AdaBoost	No-PCA	0.9457	0.9375	0.9565	0.9497	0.9389	0.9457
AdaBoost	With-PCA	0.9239	0.9293	0.9239	0.9130	0.9090	0.9198
Gradient Boosting	No-PCA	0.9497	0.9497	0.9633	0.9484	0.9484	0.9519
Gradient Boosting	With-PCA	0.9416	0.9334	0.9457	0.9416	0.9171	0.9359
XGBoost	No-PCA	0.9565	0.9511	0.9592	0.9552	0.9443	0.9533
XGBoost	With-PCA	0.9429	0.9429	0.9497	0.9416	0.9185	0.9391
Stacking	No-PCA	0.9660	0.9592	0.9565	0.9538	0.9484	0.9568
Stacking	With-PCA	0.9402	0.9402	0.9429	0.9497	0.9171	0.9380

12 Observation Questions

Which models improved most with PCA? Which did not? Why?

Strictly speaking, *no* model improved on 5-fold CV accuracy with PCA – every single one dropped, at minimum by a fraction of a point. Logistic Regression came closest to unaffected (-0.14 points CV, and actually a tiny gain on the raw test-set split), which fits the theory since linear models are exactly the kind PCA is “supposed” to help by decorrelating inputs – it just didn’t hurt here rather than clearly helping. Decision Tree suffered the most (-3.53 points), for the reason discussed in Section 10 – trees lose their ability to split on individual meaningful raw features once those features get blended into components.

Did PCA reduce variance across folds (more stable results)?

Checking the fold-wise spread (max fold – min fold) for a few models: Random Forest went from a 1.35-point spread (No-PCA) to a 3.81-point spread (With-PCA) – so PCA made it *less* stable, not more. Naive Bayes shows the same pattern even more dramatically – its With-PCA fold scores swing from 0.741 to 0.849 (a 10.9 point spread!) versus a tight 0.799-0.834 range without PCA. So for this dataset, at least with the specific

95%-variance component count we picked, PCA didn't deliver the "more stable, lower-variance folds" benefit that's often advertised – if anything the opposite happened for several models.

For high-dimensional data, was PCA beneficial in reducing overfitting?

Not really, and I think the Section 8 finding explains why – needing 48 out of 57 components to reach 95% variance means this dataset was never all that "high-dimensional" or redundant to begin with. PCA earns its overfitting-reduction reputation on datasets with hundreds or thousands of highly correlated features (image pixels, genomic data, etc.) where a handful of components really can capture most of the signal. Spambase's 57 fairly independent word-frequency features just don't compress that well, so PCA here mostly just threw away a little genuine signal along with a little noise, netting out to a small loss rather than a gain.

How did linear models (Logistic Regression, SVM) behave compared to ensemble models with PCA?

Exactly as the theory predicts – Logistic Regression and SVM were the two *least* affected by PCA (CV accuracy changes of -0.14 and -0.33 points respectively), while the ensemble/tree based models (Random Forest, AdaBoost, Gradient Boosting, Stacking) all lost noticeably more (1.4 to 2.6 points). Linear models work directly with a weighted sum of whatever features they're given, so it genuinely doesn't matter much to them whether those features are raw columns or PCA components – the model just re-learns a different weight vector. Tree-based splits, on the other hand, are much more sensitive to *which* individual features are available to split on, and lose interpretive power once features become abstract linear blends.

Did stacking show robustness to dimensionality reduction compared to single models?

Not particularly – Stacking actually had the third-largest CV accuracy drop of all 10 models (-1.88 points), behind only Decision Tree and AdaBoost. This makes sense once you remember what Stacking is built from here: two of its three base learners (Random Forest and, to a lesser extent, SVM) are exactly the kind of models that lost the most ground to PCA individually, so the ensemble inherited that same sensitivity rather than being protected by combining multiple models. Stacking's heterogeneity helps it be a consistently *strong* model overall (best or near-best in both settings), but it doesn't make it PCA-*robust* – robustness to dimensionality reduction turned out to depend on the type of base model, not on whether you're ensembling at all.

13 Report Checklist Recap

- Dataset details and preprocessing – Sections 3 and 4.
- PCA design choice with scree plot – Section 5 (95% variance target, 48 components chosen).

- Hyperparameter grids for all 10 models – Section 7.
- Fold-wise tables and average metrics, both settings – Sections 7.1 and 12.
- ROC/PR curves and confusion matrices for selected models – Section 10 (XGBoost, Decision Tree both settings, Naive Bayes, Stacking shown in full; remaining 12 combinations available in the project’s figs folder).
- Final conclusion on when PCA helps and when it doesn’t – Section 15.

14 Discussion

Going into this experiment I expected PCA to be a fairly safe, even mildly beneficial preprocessing step – that’s the common wisdom around dimensionality reduction. The actual result – every single one of 10 very different models getting *worse*, never better – was a genuinely useful thing to see empirically rather than just read about. The root cause traces back cleanly to Section 8: the 95%-variance threshold only compressed 57 features down to 48, a 16% reduction that barely qualifies as “dimensionality reduction” in the first place. PCA’s benefits (noise removal, decorrelation, faster training) scale with how aggressively you can compress; here there just wasn’t much redundancy in the Spambase feature set for PCA to exploit; each word-frequency column really is carrying its own distinct bit of spam-indicating signal.

The clearest pattern in the results is the linear-vs-tree-based split: Logistic Regression and SVM basically shrugged PCA off, while every tree-based model (single tree, bagged forest, every flavour of boosting, and the stacking ensemble that leans on RF/SVM) lost meaningfully more. This tracks with how each family actually uses its input features – a linear model just re-weights whatever axes its given, while a tree wants to split on individual, semantically meaningful features, which PCA components fundamentally aren’t.

15 Conclusion

Across 10 classifiers evaluated both with and without PCA on the Spambase dataset, **no model improved with PCA, and several – especially Decision Tree (–3.5 CV points) and AdaBoost (–2.6 points) – got noticeably worse.** The best model overall was **XGBoost without PCA** (94.79% test accuracy, 0.9874 ROC-AUC), narrowly ahead of the Stacking ensemble without PCA. The PCA setup here only reduced the feature count from 57 to 48 (95% variance threshold), which explains a lot of the outcome – this dataset simply doesn’t have enough redundancy between features for PCA to meaningfully help, so it mostly just discarded a small amount of real signal along with whatever noise it removed. Linear models (Logistic Regression, SVM) were the most PCA-tolerant, since they don’t depend on individual feature interpretability the way tree-based splits do. The practical takeaway: PCA is not a default “safe” preprocessing step – it’s worth checking the scree plot *before* committing to it, and on data that’s already fairly low-redundancy (like this one), skipping PCA and just using the original standardized features is the better call.

16 References

1. Scikit-learn Developers, “Principal Component Analysis (PCA),” <https://scikit-learn.org/stable/modules/decomposition.html#pca>
2. Scikit-learn Developers, “Ensemble methods,” <https://scikit-learn.org/stable/modules/ensemble.html>
3. XGBoost Developers, “XGBoost Documentation,” <https://xgboost.readthedocs.io/>
4. Scikit-learn Developers, “StackingClassifier,” <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html>
5. Spambase Dataset – UCI Machine Learning Repository, <https://archive.ics.uci.edu/ml/datasets/spambase>